Android Dev Summit
Extended

The state of post-quantum cryptography on Android



Mario Bodeman He/Him

Senior developer

Advocate

Yubico

#AndroidDevSummit

yubico

Post-quantum cryptography
Dangers for classics

Dangers of classical computing

“harvest now, decrypt later”
We don’t need to break encryption “now”
“Soon” is good enough
Let us capture all data we can, for later

Shor’s Algorithm

Factorization (base for classic cryptography) becomes “easy” with quantum computers
Quantum computers with enough qbits arrive “soon”

LLMs: additional risk for classics PKI

Million Dollar Challenge

“Navier-Stokes” on fluid dynamics
solved (?) by LLM controversy
Expensive token usage

NIST HAWK

LLM found flaws
NIST revoked HAWK

Marketing or Real?

Can we risk it

Post-quantum cryptography Solutions

Bigger Key Sizes

NIST: ML

Module-Lattice-Based
Key-Encapsulation Mechanism Standard (ML-KEM)
Digital Signing Algorithm Standard (ML-DSA)
Algorithm with classical means
But hard for Quantum Computers to break into

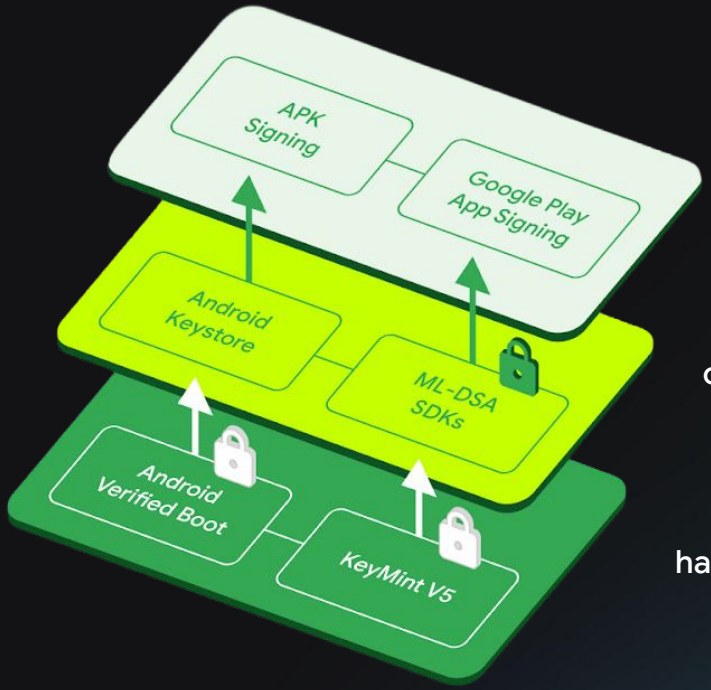
Key sizes explosion

Public key: 32B (EC) to 1.312B (ML-DSA)
Private key: 64B (EC) to 2.560B (ML-DSA)
Signature: 65B (EC) to 2.420B (ML-DSA)

https://en.wikipedia.org/wiki/Post-quantum_cryptography#Comparison

PQC today

Android 17 post-quantum chain of trust



ecosystem and distribution

operating system and APIs

hardware and silicon

The full-stack vision illustrates how Android 17 secures the entire ecosystem, establishing a continuous chain of mathematical certainty from the physical hardware boot sequence through to the global distribution of applications via Google Play.

https://blog.google/security/security-for-the-quantum-era-implementing-post-quantum-cryptography-in-android/

Testing: sign own APK

Two plus one certificates

old / current certificate from your old app version or google's keystore
New hybrid classic certificate
For older phones not understanding pqc
New hybrid pqc signature (future proofing)
ML-DSA

2x certificate lineages

Rotate keys from old to new
1x from current classic signature to new hybrid classic signature
1x from current classic signature to new hybrid pqc signature

Sign with apksigner tool

Testing: check APK signatures

Walk through all installed packages

Check certificates
⚠ Non Android 17 phones
Cannot determine version 3 certificates
I.e. cannot “see” PQC signatures

or

```
apksigner verify --print-certs-pem --verbose app-release.apk
…
V3.2 Hybrid PQC Signer: {…} key algorithm: ML-DSA
```

Testing: public key infrastructure

```
val kpg = KeyPairGenerator.getInstance(
    algo,
    "AndroidKeyStore"
)
```

Create a key pair generator

Use the build in AndroidKeyStore

Using the algorithm to be tested

- "ML-DSA"
- "ML-DSA-44"
- "ML-DSA-65"
- "ML-DSA-87"

Testing: public key infrastructure

```
val kpg = KeyPairGenerator.getInstance(
    algo,
    "AndroidKeyStore"
)

val spec = KeyGenParameterSpec.Builder(
    "alias_${algo.lowercase()}",
    KeyProperties.PURPOSE_SIGN or
KeyProperties.PURPOSE_VERIFY
)
    .setIsStrongBoxBacked(useStrongbox)
    .build()
```

Build a key spec for signing and verifying

Try to make it run on the strongbox (dedicated hardware)

Failures get thrown, be ready to catch

Testing: public key infrastructure

```
val kpg = KeyPairGenerator.getInstance(
    algo,
    "AndroidKeyStore"
)

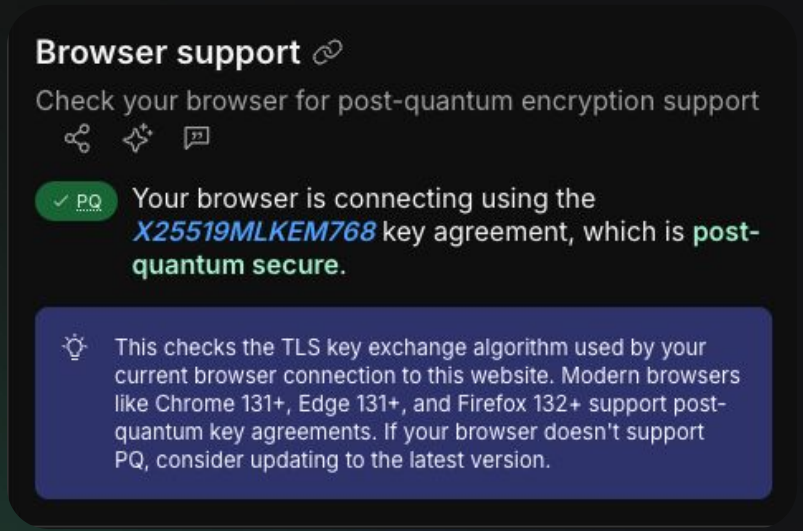
val spec = KeyGenParameterSpec.Builder(
    "alias_${algo.lowercase()}",
    KeyProperties.PURPOSE_SIGN or KeyProperties.PURPOSE_VERIFY
)
    .setIsStrongBoxBacked(useStrongbox)
    .build()

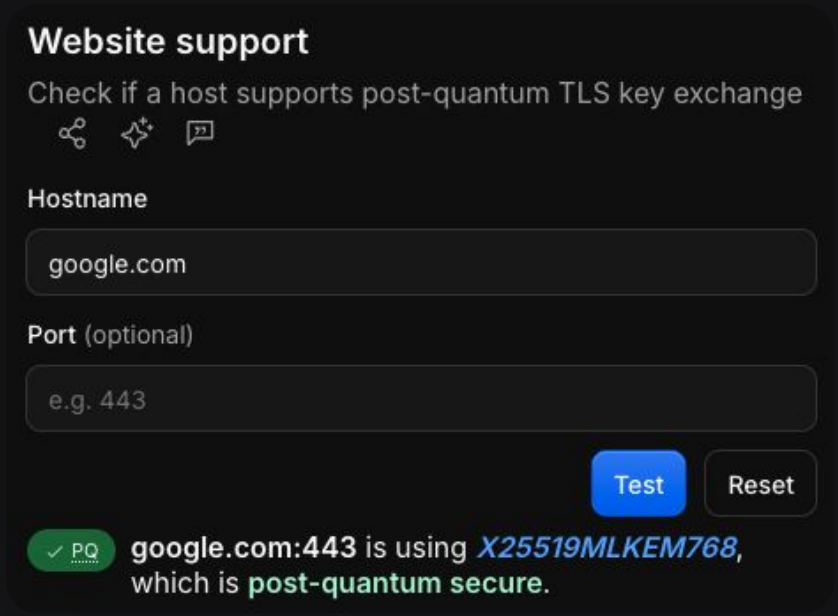
kpg.initialize(spec)
val kp = kpg.generateKeyPair()
val signer = Signature.getInstance(algo)
signer.initSign(kp.private)
signer.update(plainText.toByteArray())

val signature = signer.sign()
```

- Combine specification with generator
- Create a signer (and verifier later) for current algorithm
- Initialize signer with private key and
- Fill it with the plain text
- Finally create the signature

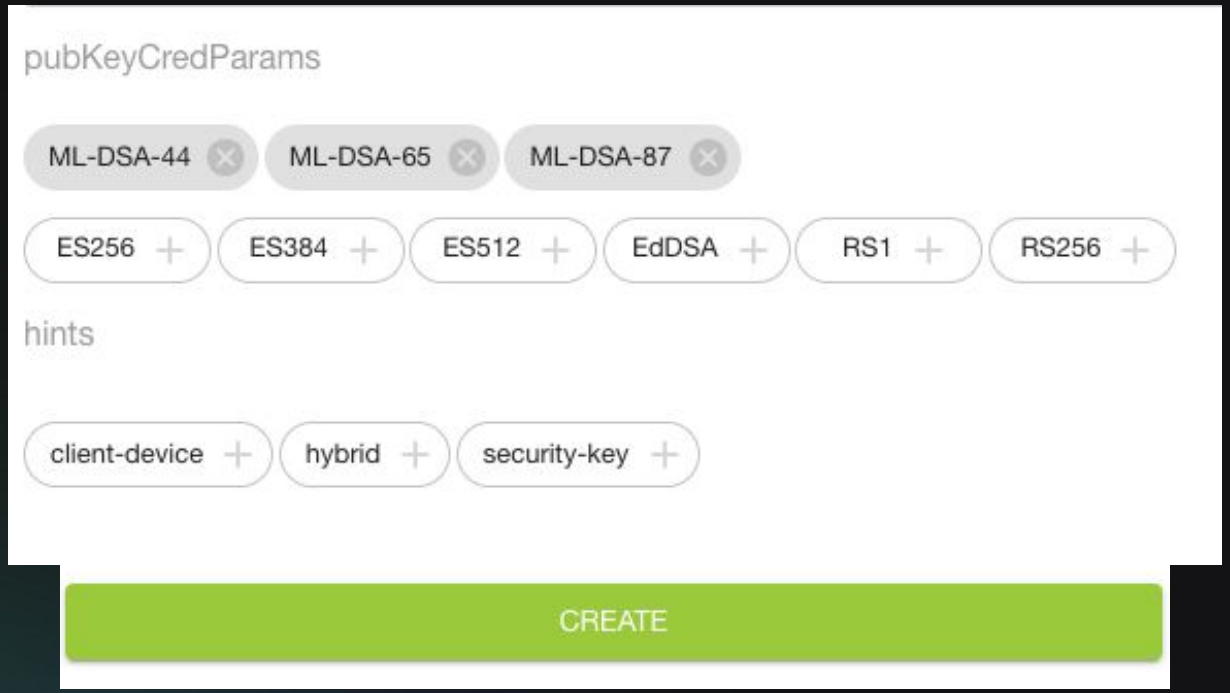
Testing: TLS





https://radar.cloudflare.com/post-quantum

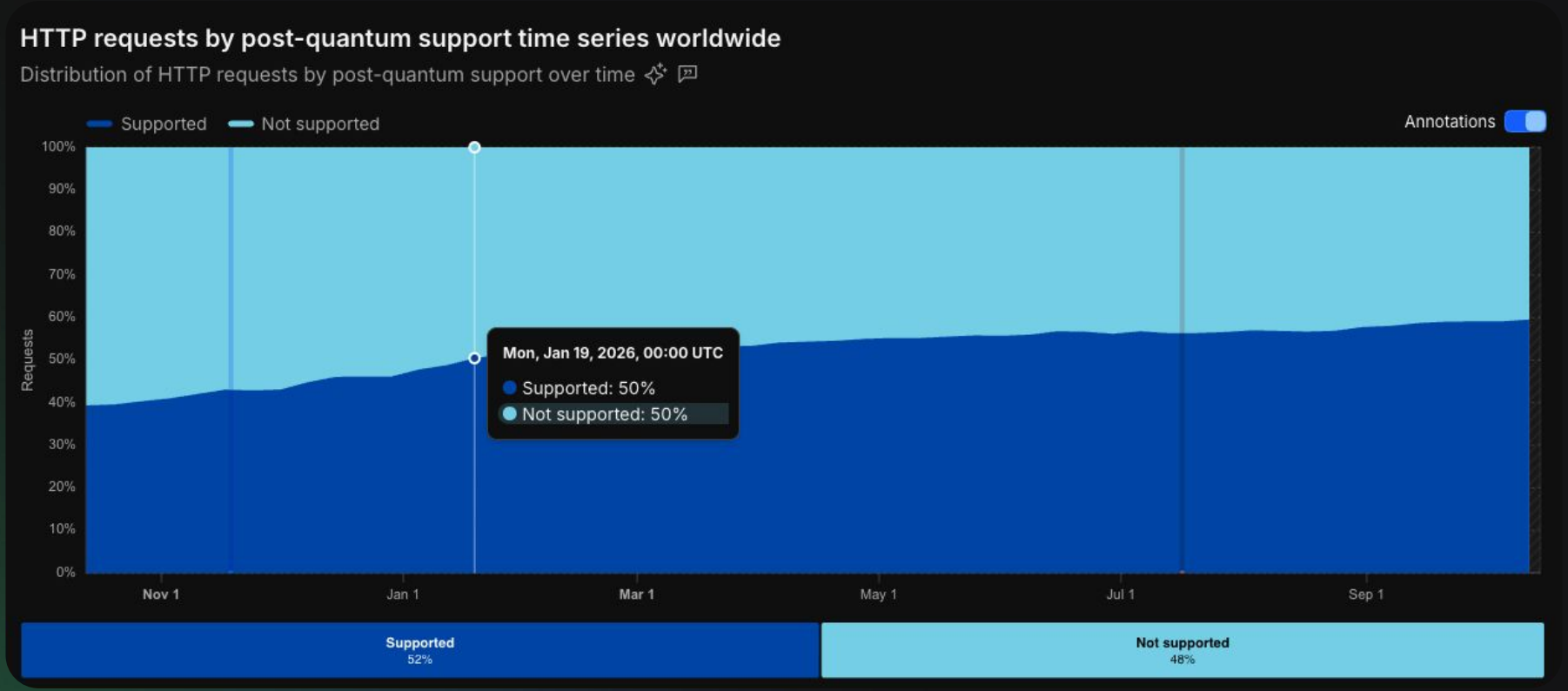
Testing: credentials (passkeys)



https://demo.yubico.com/webauthn-developers

Summary

Summary



radar.cloudflare.com/explorer?groupBy=post_quantum&dt=52w



github.com/YubicoLabs/pqc-checker-android

#AndroidDevSummit

Further Reading

FIPS-203: ML-KEM (Algorithm specification)

FIPS-204: ML-DSA (Algorithm specification)

FIPS-205: SLH-DSA (Algorithm specification)

Shor's Algorithm (Factorization)

AI Million-Dollar Prize Math Problem (Navier-Stokes fluids solved by AI controversy)

Cryptographically Weakness in NIST PQC (Anthropic and NIST not continuing with HAWK)

NIST Post Quantum Cryptography (Americans)

BUND Post Quantum Recommendations (Germans)

Bouncy Castle (3rd party crypto support)

BOTAN (© native implementation)

Signing Android APKs with apksigner (cillyvms 🫶, whoever you are)

github.com/YubicoLabs/pqc-checker-android

Thank you

